

26/05/04 Ch13 作業——class

Problem 9, 10

目錄

9.	2
題目敘述	2
題解說明	2
完整程式碼	3
10a.	4
題目敘述	4
題解說明	4
完整程式碼	4
10b.	6
題目敘述	6
題解說明	6
完整程式碼	6
A 傳值、指標、參照	8
A1. 傳值 (Pass by Value) —— 「影印給對方」	8
A2. 指標 (Pointer) —— 「給對方地址」	8
A3. 參照 (Reference) —— 「取個綽號」	8
A4. & 的雙重身分（最常搞混的點）	8
A5. 總結	8
那為什麼還需要參照？	8
B. 關鍵字 this	9
B1. this 是什麼？（它是「地址」）	9
B2. *this 是什麼？（它是「本人」）	9
B3. 為什麼要用 *this ？	9
如果你回傳 this（沒加星號）	9
如果你回傳 *this（有加星號）	9
B4. 總結	9
C. Function with Pointer and Reference	10
C1. 使用指標 Pointer (* 與 &)	10
C2. 使用引用 Reference (&)	10
C3 快速對照表	10

9.

題目敘述

9. 某業務員銷售二款手機，試建立一個 CSale() 建構子，可接收一個 char 型態的變數 id，以及兩個 int 型態的變數 q 與 p，並為物件的資料成員-產品編號 (id)、銷售數量 (quantity)、單價 (price) 設值。請依序完成下列各題：

- (a) 撰寫 show() 函數，可印出產品的編號、銷售數量、單價以及該產品的銷售額 (銷售額=銷售數量*單價)。
- (b) 撰寫 compare() 函數，以指向物件的指標為引數，印出銷售額較高的產品。
- (c) 撰寫 few() 函數，使得該函數的傳回值是指向銷售額較少物件之參照。

請參考下面的執行結果：

- (a)
產品 A：銷售數量=10 個，單價=34699 元，銷售額=346990 元
產品 B：銷售數量=12 個，單價=40400 元，銷售額=484800 元
- (b)
產品 B 的銷售額較高
- (c)
產品 A 的銷售額較少

題解說明

(b) 以指向物件的**指標**為引數，**印出銷售額較高的產品**：

```
void compare(CSale* other)
{
    if (this->sale > other->sale) { cout << "產品 " << this->id << " 的銷售額較高" << endl; }
    else { cout << "產品 " << other->id << " 的銷售額較高" << endl; }
}
```

(c) 該函數的**傳回值**是指向銷售額較少物件之**參照**：

```
CSale& few(CSale& other)
{
    //if ((*this).sale), *this 可省略
    if (sale < other.sale) { return *this; }
    else { return other; }
}
```

```
(a)
產品 A：銷售數量=10 個，單價=34699 元，銷售額=346990 元
產品 B：銷售數量=12 個，單價=40400 元，銷售額=484800 元

(b)
產品 B 的銷售額較高

(c)
產品 A 的銷售額較少

Program finished with exit code 0
Press any key to exit
```

完整程式碼

```
1 #include <iostream>
2 using namespace std;
3
4 class CSale
5 {
6     private:
7         char id;
8         int quantity;
9         int price;
10        int sale;
11
12    public:
13        // 建構子
14        CSale(char i, int q, int p) : id(i), quantity(q), price(p), sale(q * p)
15        {
16            // 沒有其他的事要做，空白
17        }
18
19        char Get_id()    { return id; }
20
21        // (a) 印出產品資訊
22        void show()
23        {
24            cout << "產品 " << id << ": "
25                << "銷售數量=" << quantity << " 個, "
26                << "單價=" << price << " 元, "
27                << "銷售額=" << sale << " 元" << endl;
28        }
29
30        // (b) compare() 函數：已指向物件的指標為引數，印出銷售額較高的產品。
31        void compare(CSale* other)
32        {
33            if (this->sale > other->sale) { cout << "產品 " << this->id << " 的銷售額較高" << endl;}
34            else { cout << "產品 " << other->id << " 的銷售額較高" << endl;}
35        }
36
37        // (c) few() 函數：該函數的傳回值是指向銷售額較少物件之參照
38        CSale& few(CSale& other)
39        {
40            if (sale < other.sale) { return *this; }
41            else { return other; }
42        }
43 };
44
45 int main()
46 {
47     CSale a('A', 10, 34699);
48     CSale b('B', 12, 40400);
49
50     cout << "(a)" << endl;
51     a.show();
52     b.show();
53
54     cout << "\n(b)" << endl;
55     a.compare(&b);
56
57     cout << "\n(c)" << endl;
58     cout << "產品 " << (a.few(b)).Get_id() << " 的銷售額較少" << endl;
59
60     return 0;
61 }
```

10a.

題目敘述

10. 某業務員本月銷售二款汽車，請依照要求完成下列各題：

- (a) 資料成員 id (車子代號)、quantity (銷售數量)、price (單價) 與 sale (銷售額) 成員的存取屬性為 private，利用 set_data() 函數設定資料成員的內容。
- (b) 將 set_data() 函數寫成友誼函數，同時也使它們可以多載。

請參考下面的執行結果：

車款：ix M60，銷售數量 3 台，單價=638 萬元，銷售額=1914 萬元

車款：x5 M Sport，銷售數量 5 台，單價=392 萬元，銷售額=1960 萬元

題解說明

照題意模擬即可。

```
車款：ix M60，銷售數量 3 台，單價=638 萬元，銷售額=1914 萬元
車款：x5 M Sport，銷售數量 5 台，單價=392 萬元，銷售額=1960 萬元

Program finished with exit code 0
Press any key to exit
```

完整程式碼

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class CCar
6 {
7     private:
8         string id;
9         int quantity;
10        int price;
11        int sale;
12
13    public:
14        // (a) 成員函數 set_data
15        void set_data(string i, int q, int p)
16        {
17            id = i;
18            quantity = q;
19            price = p;
20            sale = q * p;
21        }
22
23        void show()
24        {
25            cout << "車款: " << id << ", "
26                << "銷售數量 " << quantity << " 台, "
27                << "單價=" << price << " 萬元, "
28                << "銷售額=" << sale << " 萬元" << endl;
29        }
30 };
```

```
31
32 int main()
33 {
34     CCar car1;
35     CCar car2;
36
37     car1.set_data("ix M60", 3, 638);
38     car2.set_data("x5 M Sport", 5, 392);
39
40     car1.show();
41     car2.show();
42
43     return 0;
44 }
45
```

10b.

題目敘述

10. 某業務員本月銷售二款汽車，請依照要求完成下列各題：

(a) 資料成員 `id` (車子代號)、`quantity` (銷售數量)、`price` (單價) 與 `sale` (銷售額) 成員的存取屬性為 `private`，利用 `set_data()` 函數設定資料成員的內容。

(b) 將 `set_data()` 函數寫成友誼函數，同時也使它們可以多載。

請參考下面的執行結果：

車款：ix M60，銷售數量 3 台，單價=638 萬元，銷售額=1914 萬元

車款：x5 M Sport，銷售數量 5 台，單價=392 萬元，銷售額=1960 萬元

題解說明

友誼函數：可在 `class` 外部，實作函數內容，不需加上 `friend` 或 `::` 作用域運算子。

```
friend void set_data(CCar& car, string i, int q, int p);
```

在程式碼中，`set_data` 是外部函數，但因為在 `CCar` 內被宣告為 `friend` 因此：

- 直接存取：它可以直接寫 `car.id` 或 `car.price`，不必透過公有介面。
- 傳址呼叫：使用 `CCar& car` (引用) 來直接修改傳入的那台車，而不是修改副本。

同 10a 結果

```
車款：ix M60，銷售數量 3 台，單價=638 萬元，銷售額=1914 萬元
車款：x5 M Sport，銷售數量 5 台，單價=392 萬元，銷售額=1960 萬元

Program finished with exit code 0
Press any key to exit
```

完整程式碼

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class CCar
6 {
7     private:
8         string id;
9         int quantity;
10        int price;
11        int sale;
12
13    public:
14        // (b) 宣告友誼函數與多載
15        friend void set_data(CCar& car, string i, int q, int p);
16
17        void show()
18        {
19            cout << "車款: " << id << ", "
20                 << "銷售數量 " << quantity << " 台, "
```

```
21         << "單價=" << price << " 萬元, "
22         << "銷售額=" << sale << " 萬元" << endl;
23     }
24 };
25
26 void set_data(CCar& car, string i, int q, int p)
27 {
28     car.id = i;
29     car.quantity = q;
30     car.price = p;
31     car.sale = q * p;
32 }
33
34 int main()
35 {
36     CCar car1, car2;
37
38     set_data(car1, "ix M60", 3, 638);
39     set_data(car2, "x5 M Sport", 5, 392);
40
41     car1.show();
42     car2.show();
43
44     return 0;
45 }
46
```

A 傳值、指標、參照

想像你有一本筆記本（這就是物件/變數）。

A1. 傳值 (Pass by Value) —— 「影印給對方」

函數收到的是一份副本，怎麼改都不影響原本。

- 缺點：物件很大時，影印很花時間與記憶體。

A2. 指標 (Pointer) —— 「給對方地址」

你給對方一張紙條，上面寫著筆記本的存放位置。

- &（取址）：取得紙條上的地址
- *（解參考）：拿著地址，找到本人

```
int a = 5;
int* p = &a; // p 存的是 a 的地址
*p = 10;     // 解參考，直接改 a 本人
```

-> 只是 struct/class 的語法糖，等價於 (*ptr).member，跟「找到本人」這個動作本身無關。

- 危險性：地址可以是空的（nullptr），存取前最好檢查。

A3. 參照 (Reference) —— 「取個綽號」

直接幫筆記本取另一個名字，它就是本人，不是副本也不是地址。

```
int a = 5;
int& r = a; // r 就是 a，同一個東西
r = 10;     // a 也變成 10
```

- 安全性：宣告時就必須綁定，不能為空，不能改指向別人。

A4. & 的雙重身分（最常搞混的點）

```
int a = 5;
int* p = &a; // 取址運算子：取得 a 的地址
int& r = a;  // 參照宣告符：r 是 a 的別名
```

同一個符號，位置不同，意思完全不同。

A5. 總結

概念	你手上拿著什麼	能改本人?	效能	可為空?
傳值	副本	✗	慢	—
指標	地址	✓	快	✓ (危險)
參照	別名	✓	快	✗ (安全)

那為什麼還需要參照？

指標能做的參照大多也能做，但：

1. **語法乾淨**：用 `.` 就好，不用滿版 `*`
 2. **保證有效**：不能為空、不能懸空
 3. **支援鏈式呼叫**：回傳參照等於把「本人的控制權」直接傳下去，中間沒有任何複製
-

B. 關鍵字 `this`

在 C++ 的物件導向程式設計中，`this` 是一個非常核心的概念。我們可以從它的「身分」以及加上 `*`（星號）之後的「行為」來拆解：

B1. `this` 是什麼？（它是「地址」）

當你在類別（Class）的成員函數內部時，`this` 是一個隱含的指標。它儲存了目前正在執行這個函數的物件在記憶體中的地址。

- **比喻**：如果你是一間房子的「管家」（成員函數），`this` 就是你手上拿著的那間房子的「門牌地址」。
- **特性**：
 - 它永遠指向「自己」。
 - 它的型別是 `CSale*`（指向該類別的指標）。

B2. `*this` 是什麼？（它是「本人」）

當你在 `this` 前面加上 `*`（解引用運算子，Dereference），意思就是：「順著這個地址進去，找到那個物件本人」。

- **比喻**：這就像是你順著門牌地址推門進去，看到的不再是地址紙條，而是房子建築物本身。
 - **在程式中的意義**：
 - 它的型別是 `CSale`（物件本身）。
 - 當你的函數回傳型別是參照（`CSale&`）時，你必須回傳 `*this`，代表「我把這個物件本人（而非地址）傳回去給呼叫者」。
-

B3. 為什麼要用 `*this` ？

我們用你的 `few()` 函數來做圖解對比：

如果你回傳 `this`（沒加星號）

這會發生編譯錯誤。因為你的函數承諾要回傳一個「物件參照（本尊）」，但你卻丟出了一個「指標（地圖）」。這就像是快遞員說要送貨物（物件）給你，結果只給了你一張寫著倉庫地址的紙條（指標）。

如果你回傳 `*this`（有加星號）

這完全符合要求。你告訴程式：「請把我現在所在的這個物件直接傳回去」。

```
1 CSale& few(CSale& other)
2 {
3     if ((*this).sale < other.sale)
4     {
5         return *this; // 「我」比「你」小，所以傳回「我本人」
6     }
7     else
8     {
9         return other; // 「你」比「我」小，所以傳回「你本人」
10    }
11 }
```

B4. 總結

- `this`：地址（指標），用來找路。存取成員時用 `this->ID`。
- `*this`：物件（本尊），用來當作傳回值，代表物件自己。

這就是為什麼在你的 Problem 9c 中，為了滿足「傳回參照」的要求，必須寫成 `return *this;` 的原因。

C. Function with Pointer and Reference

以下用一個簡單的「更換姓名」範例，讓你一眼看出指標（Pointer）與引用（Reference）在語法上的差別：

C1. 使用指標 Pointer (* 與 &)

這是傳統 C 語言風格，呼叫時必須明確「取址」，函數內必須明確「解引用」。

```
1 struct User {
2     string name;
3 };
4 // 參數用 * 接收地址void changeName(User* u) {
5     u->name = "Alice"; // 使用 -> 存取成員
6 }
7 int main() {
8     User person = {"Bob"};
9     // 呼叫時必須加 & 取得地址
10    changeName(&person);
11 }
```

C2. 使用引用 Reference (&)

這是 C++ 特有的語法，看起來像傳入一般變數，但實際上操作的是同一個物件。

```
1 struct User {
2     string name;
3 };
4 // 參數用 & 宣告為引用void changeName(User& u) {
5     u.name = "Alice"; // 使用 . 存取成員，像操作一般變數
6 }
7 int main() {
8     User person = {"Bob"};
9     // 呼叫時直接傳入物件，不需加符號
10    changeName(person);
11 }
```

C3 快速對照表

特性	指標 Pointer (*)	引用 Reference (&)
定義方式	User* u	User& u
呼叫方式	func(&person) (需取址)	func(person) (直接傳)
內部操作	u->name (需解引用)	u.name (直接用)
空值安全性	可以是 nullptr (需檢查)	不能為空 (較安全)
直觀程度	像在操作「地址」	像在操作「本體」的別名

📌 建議：在 C++ 中，如果不需要處理「空值（NULL）」或「更換指向對象」，優先使用引用 (&)，程式碼會更乾淨且安全。